

4장 상태관리 및 데이터 연동

이번 장은 React 애플리케이션에서 핵심이 되는 상태 관리와 데이터 연동을 심도 있게 다루며, 이벤트 처리 메커니즘을 통한 사용자 입력 제어, `useState·useEffect` 혹은 활용한 동적 상태 갱신과 비동기 처리, Context API를 통한 전역 상태 공유까지 학습함으로써 실무 애플리케이션에서 일관된 데이터 흐름과 효율적인 상태 관리 구조를 구현할 수 있는 기초와 응용 능력을 동시에 습득한다.

4.1 이벤트 처리 메커니즘

React의 이벤트 처리 메커니즘은 DOM 이벤트를 캡슐화한 합성 이벤트(Synthetic Event)를 기반으로 동작하며, `onClick·onChange`와 같은 표준 이벤트 속성, 이벤트 핸들러 함수 연결 방식, 입력값에 따른 상태 갱신, 상위-하위 컴포넌트 간 함수 전달 구조까지 이해함으로써 UI와 데이터 상태를 자연스럽게 연결하는 기반을 마련한다.

(1) `onClick`, `onChange` 등의 이벤트 구조

React 이벤트는 W3C 표준을 따르는 합성 이벤트 시스템으로 동작하며, `onClick·onChange·onSubmit` 등과 같은 속성을 통해 이벤트를 처리하여 브라우저 간 일관된 동작과 성능 최적화를 동시에 제공한다.

① 핵심 개념

- React는 브라우저의 네이티브 이벤트를 감싸 `SyntheticEvent` 라는 통일된 객체로 제공해 `e.target`, `e.currentTarget`, `e.preventDefault()`, `e.stopPropagation()` 등의 공통 API를 사용하게 한다.
- 내부적으로는 이벤트 위임(일괄 리스너 등록 — 보통 루트 컨테이너에 등록) 방식으로 많은 DOM 요소에 개별 리스너를 붙이지 않아도 되게 처리한다(구현 세부는 React 버전·렌더러에 따라 다름).
- React의 `onChange` 는 `<input>` / `<textarea>` 의 경우 사용자가 값을 입력할 때마다(사실상 `input` 이벤트처럼) 호출된다 — 브라우저 `change` 와 일대일 대응이 아니라 React가 정한 의미를 가진다.

② 주요 속성 및 메서드

- `e.target` : 이벤트가 발생한 실제 DOM 요소(입력값을 읽을 때는 `e.target.value` 사용).
- `e.currentTarget` : 이벤트 리스너가 바인딩된 요소(버블링/위임을 고려할 때 유용).
- `e.preventDefault()` : 폼 제출 등 브라우저 기본 동작 방지.
- `e.stopPropagation()` : 이벤트 전파(버블링) 중지.

③ 예제

```
function Example(){
  const handleClick = (e) => {
    console.log('type:', e.type); // "click"
    // 필요한 값은 즉시 복사
    const tag = e.currentTarget.tagName;
    console.log(tag);
  };
  return <button onClick={handleClick}>클릭</button>;
}
```

④ 주의사항

- 이벤트 객체를 비동기 콜백(예: `setTimeout`)에서 사용하려면 필요한 속성 (`e.target.value`)을 즉시 지역변수로 복사해서 사용하세요. (과거 React에서는 이벤트 객체가 재사용될 수 있었고, 안전을 위해 값을 복사하는 습관이 유용함.)
- `onChange` 는 React에서 controlled input에 쓰이는 표준이며, `value` 와 함께 쓰지 않으면 예상치 못한 동작이 발생할 수 있음.

(2) 이벤트 핸들러 선언 및 연결 방식

이벤트 핸들러는 일반 함수 또는 화살표 함수로 선언되어 JSX 요소의 속성에 직접 연결되며, 인자 전달과 this 바인딩 문제를 해결해 컴포넌트의 동작을 깔끔하게 정의할 수 있는 핵심적인 구조를 형성한다.

① 함수 선언 방식

- 함수형 컴포넌트

```
const handleClick = (e) => { /* ... */ };
<button onClick={handleClick}>...</button>
```

- 클래스형 컴포넌트

```
class C extends React.Component {
  handleClick = (e) => { /* arrow property: this 바인딩 해결 */ }
  render(){ return <button onClick={this.handleClick}>OK</button> }
}
```

또는 생성자에서 `this.handleClick = this.handleClick.bind(this)` 로 바인딩.

② 파라미터 전달 방법

- 화살표 함수로 래핑 (가장 흔함)

```
<button onClick={() => handleDelete(id)}>삭제</button>
```

- `bind` 사용

```
<button onClick={handleDelete.bind(null, id)}>삭제</button>
```

- 주의: `onClick={handleDelete(id)}` 처럼 직접 호출하면 렌더링 시 함수가 즉시 실행되므로 피해야 함.

③ 성능 고려

- JSX 내부에 `() => fn(arg)` 를 자주 사용하면 매 렌더링마다 새 함수가 생성되어 하위 컴포넌트에 prop으로 전달될 때 불필요한 재렌더링을 유발할 수 있음.
- 해결책: `useCallback` 으로 핸들러를 메모이제이션하거나, 하위 컴포넌트를 `React.memo` 로 감싸 비교를 통해 불필요 렌더를 막음.

```
const onRemove = useCallback((id) => setTodos(t => t.filter(x => x.id !== id)), []);
```

④ 클래스의 this 문제

- 클래스에서 `handleClick()` 을 메서드로 선언하면 `this` 가 undefined가 될 수 있으니 바인딩 필요.
- 화살표 메서드(`handleClick = () =>`)를 사용하면 바인딩 문제 없음.

(3) 사용자 입력에 따른 상태 변경 실습

입력창이나 버튼 클릭 이벤트 발생 시 `useState`로 관리되는 상태 값을 업데이트하여 화면을 즉시 갱신하고, 이를 통해 React의 단방향 데이터 흐름과 UI 반영 원리를 실습할 수 있다.

① Controlled Component (권장 패턴)

- 입력의 값(`value`)을 컴포넌트 상태로 보관하고, `onChange` 에서 이 상태를 갱신하면 UI가 상태를 반영한다.
- 예:

```
function NameInput(){
  const [name, setName] = useState('');
  return (
    <input value={name}
      onChange={(e) => setName(e.target.value)}
      placeholder="이름 입력"
    />
  );
}
```

- 장점: 입력값 검증, 포맷 변환, 폼 초기화가 쉬움.

② Uncontrolled Component (ref 사용)

- 간단한 경우 DOM ref로 값을 읽는 방법도 있음:

```
const ref = useRef();
<input ref={ref} />
```

```
// onSubmit에서 ref.current.value 읽기
```

- 그러나 복잡한 폼에서는 controlled가 더 안전.

③ 폼 제출 제어

- `<form onSubmit={handleSubmit}>` 에서 `e.preventDefault()` 를 사용하여 페이지 리로드를 막고, 상태로 값 처리:

```
function handleSubmit(e){  
  e.preventDefault();  
  // 상태값 사용  
}
```

④ 실시간 유효성 검사 / 즉시 피드백

- `onChange` 에서 유효성 검사를 하여 에러 메시지를 상태로 저장하고 UI에 보여준다.
- 예: 이메일 포맷 검사, 최소 길이 검사 등.

⑤ 비동기 입력 처리 (디바운스 예시 — 검색박스)

- 사용자가 타이핑할 때마다 API 콜을 하지 않도록 디바운스:

```
const [q, setQ] = useState('');  
useEffect(() => {  
  const id = setTimeout(() => { doSearch(q); }, 300);  
  return () => clearTimeout(id);  
}, [q]);
```

- 주의: `doSearch` 는 `useCallback` 으로 고정하거나 의존성에 맞춰 관리.

(4) 함수 전달 구조 이해

상위 컴포넌트에서 정의한 함수를 props를 통해 하위 컴포넌트에 전달하여 하위 컴포넌트의 이벤트 발생이 상위 상태를 변경하도록 설계함으로써 컴포넌트 간 데이터 흐름과 제어 권

한을 명확히 이해할 수 있다.

① Lifting State Up 패턴

- 상태가 여러 자식에 의해 필요하면 그 상태를 공통 부모로 올리고, 부모는 상태 변경 함수(예: `addItem`, `removeItem`)를 정의해 자식에 `props` 로 전달한다.
- 예:

```
function Parent(){
  const [items, setItems] = useState([]);
  const addItem = (t) => setItems(s => [...s, { id:Date.now(), text:t }]);
  return <ChildForm onAdd={addItem} />;
}
function ChildForm({ onAdd }) { /* onAdd('hello') */ }
```

② 인자(값) 전달 방식

- 하위에서 상위로는 **값(value)** 또는 **식별자(id)** 를 전달하는 것이 일반적(예: `onRemove(id)`).
- 가능한 경우 이벤트 객체 대신 필요한 값만 전달하라 (`onChange` 에서 `e.target.value` 대신 `onChangeValue(value)` 호출) — 이렇게 하면 하위 컴포넌트가 상위의 내부 구현(이벤트 객체)에 종속되지 않음.

③ prop drilling 문제와 해결

- 여러 중첩 컴포넌트를 통해 동일 콜백을 전달하면 코드가 지저분해짐 → 해결책:
 - Context API로 전역/중간 단계 생략
 - 컴포지션(Children as Function / render props)
 - 상태관리 라이브러리 (Zustand, Redux 등)

④ 성능 고려 및 메모이제이션

- 부모가 `onChange` 함수를 재생성하면 `React.memo` 로 감싼 자식도 재렌더링됨. 이를 막으려면 부모에서 `useCallback` 을 사용:

```
const onRemove = useCallback((id) => setItems(s => s.filter(x => x.id !== id)), []);
```

- 그러나 무분별한 `useCallback` 사용은 코드 복잡성만 늘리므로, 실제로 성능 이슈가 증명될 때 적용하라.

⑤ 패턴 예제 — Todo

- 부모: `todos`, `addTodo`, `removeTodo`, `toggleTodo` 보유
- 하위 `AddTodo` : `onAdd` prop으로 문자열 받음
- 하위 `TodoItem` : `onRemove`, `onToggle` prop 받아 UI 이벤트 발생 시 호출

⑥ 테스트와 타입 안정성

- Prop으로 전달되는 함수의 시그니처를 명확히 문서화/타입화(TypeScript)하면 하위 컴포넌트 작성자가 올바른 인자를 넘겨줄 수 있음.
- 테스트: RTL(React Testing Library)에서 `fireEvent.click` 호출 후 상위 상태 변화가 일어나는지 검증.

(5) 이벤트 처리 메커니즘 실습

React의 합성 이벤트(Synthetic Event), `onClick/onChange/onSubmit` 사용법, 이벤트 핸들러 선언·연결, 상위→하위 함수 전달 구조를 실습으로 익히는 예제 앱입니다. 간단한 Todo 앱을 통해 입력 이벤트와 버튼 클릭 이벤트로 상태를 변경하고, 하위 컴포넌트에서 상위 함수 호출로 상태를 제어하는 패턴을 보여줍니다.

프로젝트 디렉터리 구조

```
ex01-event-handling/  
├─ package.json  
├─ vite.config.js  
├─ index.html  
├─ public/  
│   └─ favicon.ico
```

```
├─ src/
│   ├─ main.jsx
│   ├─ App.jsx
│   ├─ styles.css
│   └─ components/
│       ├─ TodoApp.jsx
│       ├─ AddTodo.jsx
│       ├─ TodoList.jsx
│       └─ TodoItem.jsx
└─ README.md
```

① 빠른 시작 명령 (터미널 명령어)

```
# 1) 프로젝트 생성 폴더로 이동한 뒤
# (폴더명이 ex01-event-handling일 때)
npm init -y
# package.json을 아래 내용으로 덮어쓰기(또는 수동 편집)

# 2) 의존성 설치
npm install react react-dom
npm install -D vite

# 3) 개발 서버 실행
npx vite
# 또는 package.json scripts 후
npm run dev
```

(아래 package.json 스니펫을 사용하면 npm install 후 npm run dev 로 개발 서버 실행 가능)

② package.json

```
{
  "name": "ex01-event-handling",
  "version": "0.0.1",
  "private": true,
```



```

"type": "module",
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
},
"dependencies": {
  "react": "^18.2.0",
  "react-dom": "^18.2.0"
},
"devDependencies": {
  "vite": "^5.4.8"
}
}

```

③ vite.config.js

```

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()]
});

```

④ index.html

```

<!doctype html>
<html lang="ko">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>ex01 - Event Handling</title>
  </head>
  <body>
    <div id="root"></div>

```

```

    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

⑤ src/main.jsx

```

import React from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import './styles.css'

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)

```

⑥ src/App.jsx

```

import React from 'react'
import TodoApp from './components/TodoApp'

export default function App(){
  return (
    <div className="app">
      <header>
        <h1>ex01 - 이벤트 처리 실습 (Todo)</h1>
        <p>onChange, onClick, onSubmit, 그리고 상위→하위 함수 전달 예제입니
다.</p>
      </header>
      <main>
        <TodoApp />
      </main>
    </div>
  )
}

```

```
)  
}
```

⑦ src/styles.css

```
:root{ --bg:#f6f8fa; --card:#fff; --muted:#666; }  
body{ font-family: Inter, Roboto, system-ui, -apple-system; margin:0; background:var(--bg); color:#111 }  
.app{ max-width:800px; margin:40px auto; padding:24px; }  
header h1{ margin:0 0 6px 0 }  
.card{ background:var(--card); border-radius:8px; padding:16px; box-shadow:0 6px 18px rgba(0,0,0,.06) }  
.form-row{ display:flex; gap:8px; margin-bottom:12px }  
.input{ flex:1; padding:8px 10px; border:1px solid #ddd; border-radius:6px }  
.btn{ padding:8px 12px; border:none; background:#2274e1; color:#fff; border-radius:6px; cursor:pointer }  
.btn:disabled{ opacity:.6 }  
.todo-list{ margin-top:12px }  
.todo-item{ display:flex; justify-content:space-between; padding:10px; border-bottom:1px solid #f0f0f0 }  
.todo-item:last-child{ border-bottom:none }  
.small{ font-size:13px; color:var(--muted) }
```

⑧ src/components/ToDoApp.jsx

```
import React, { useState, useCallback } from 'react'  
import AddTodo from './AddTodo'  
import TodoList from './TodoList'  
  
// 부모 컴포넌트: 상태 보유 및 하위로 함수 전달  
export default function ToDoApp(){  
  const [todos, setTodos] = useState([  
    { id: 1, text: 'React 학습', done: false },  
    { id: 2, text: '이벤트 처리 예제 만들기', done: false }  
  ])  
}
```

```

// 추가 함수: 하위 컴포넌트에서 호출됨
const addTodo = useCallback((text) => {
  if(!text || !text.trim()) return;
  setTodos((s) => [
    ...s,
    { id: Date.now(), text: text.trim(), done: false }
  ])
}, [])

// 삭제 함수: 하위 컴포넌트에서 호출됨
const removeTodo = useCallback((id) => {
  setTodos((s) => s.filter(t => t.id !== id))
}, [])

// 완료 토글 함수 (파라미터 전달 예시)
const toggleDone = useCallback((id) => {
  setTodos((s) => s.map(t => t.id === id ? { ...t, done: !t.done } : t))
}, [])

return (
  <div className="card">
    <AddTodo onAdd={addTodo} />
    <div className="small">총 항목: {todos.length}</div>
    <TodoList todos={todos} onRemove={removeTodo} onToggle={toggleDone} />
  </div>
)
}

```

⑨ src/components/AddTodo.jsx

```

import React, { useState } from 'react'

// 하위 컴포넌트: 입력과 제출 처리, 상위 onAdd 호출
export default function AddTodo({ onAdd }) {
  const [text, setText] = useState('')

```

```

// onChange 핸들러: 합성 이벤트의 event.target.value 사용
const handleChange = (e) => {
  setText(e.target.value)
}

// onSubmit 핸들러: 폼 제출 기본 동작 막기
const handleSubmit = (e) => {
  e.preventDefault() // 브라우저 기본 폼 제출을 막음
  onAdd(text)
  setText('')
}

return (
  <form onSubmit={handleSubmit} className="form-row" aria-label="add-todo-form">
    <input
      className="input"
      value={text}
      onChange={handleChange} // onChange 연결
      placeholder="할 일을 입력하고 Enter 또는 추가 버튼"
      aria-label="새 할 일 입력"
    />
    <button type="submit" className="btn" disabled={!text.trim()}>추가</button>
  </form>
)
}

```

⑩ src/components/ToDoList.jsx

```

import React from 'react'
import TodoItem from './TodoItem'

export default function ToDoList({ todos, onRemove, onToggle }) {
  if(!todos.length) return <div className="small">할 일이 없습니다.</div>
  return (
    <div className="todo-list">

```

```

      {todos.map(t => (
        <TodoItem key={t.id} todo={t} onRemove={onRemove} onToggle={on
Toggle} />
      ))}
    </div>
  )
}

```

⑪ src/components/TodoItem.jsx

```

import React from 'react'

export default function TodoItem({ todo, onRemove, onToggle }) {
  const { id, text, done } = todo

  // onClick에 파라미터 전달하는 방법: 화살표 함수로 래핑
  const handleRemove = () => onRemove(id)
  const handleToggle = (e) => {
    // 이벤트 객체가 필요하면 e 사용 가능 (예: e.stopPropagation())
    onToggle(id)
  }

  return (
    <div className="todo-item" role="listitem">
      <div>
        <input type="checkbox" checked={done} onChange={handleToggle}
aria-label={`완료 ${text}`} />
        <span style={{ marginLeft: 8, textDecoration: done ? 'line-through' : 'none' }}>{text}</span>
      </div>
      <div>
        <button onClick={handleRemove} className="btn">삭제</button>
      </div>
    </div>
  )
}

```

⑫ 실행 및 테스트

1. `npm install` (package.json 준비 후)
2. `npm run dev` → 브라우저에서 `http://localhost:5173` 열기
3. 입력 상자에 텍스트 입력 → Enter 또는 추가 버튼 클릭 → 항목이 추가되는지 확인
4. 체크박스 클릭 → 체크 상태로 토글되는지 확인
5. 삭제 버튼 클릭 → 항목이 삭제되는지 확인

4.2 useState, useEffect Hook 활용

React 혹은 클래스형 컴포넌트의 한계를 보완하기 위해 도입된 함수형 컴포넌트 전용 기능으로, `useState`는 컴포넌트 내 상태를 선언·갱신하는 핵심 도구이며 `useEffect`는 데이터 요청·DOM 조작·구독과 같은 부수 효과를 관리하는 메커니즘으로, 두 Hook의 조합은 상태와 라이프사이클을 직관적으로 제어할 수 있게 해준다.

(1) Hook의 정의와 사용 이유

Hook은 함수형 컴포넌트에서도 상태 관리와 생명주기 로직을 사용할 수 있게 만든 기능으로, 코드 재사용성과 가독성을 높이고 클래스 문법에 의존하지 않는 선언적 컴포넌트 설계를 가능하게 한다.

- React 16.8에서 도입
- 함수형 컴포넌트에서도 상태와 사이드 이펙트(부수효과) 관리 가능
- 코드 재사용성 증가: 커스텀 Hook을 통해 로직을 모듈화
- 클래스 문법의 `this` 바인딩 문제 해결
- 함수 중심의 선언적 프로그래밍 스타일 강화

(2) useState 기반 상태 초기화 및 갱신

useState는 초기값을 지정하고 상태 변수와 setter 함수를 반환하여 상태를 선언하며, setter를 호출할 때마다 컴포넌트가 재렌더링되어 UI를 최신 상태로 동기화한다.

```
const [count, setCount] = useState(0);
```

- `count`: 상태 변수 (현재 값 저장)
- `setCount`: setter 함수 (상태 갱신 → 리렌더링 발생)
- 인자는 초기값 (`0`)
- setter를 호출하면 React는 해당 상태를 갱신 후 컴포넌트를 리렌더링

예시: 버튼 클릭 시 카운터 증가

```
<button onClick={() => setCount(count + 1)}>+1</button>
```

(3) useEffect를 통한 비동기 로직 처리

useEffect는 렌더링 이후 실행되는 부수 효과 로직을 정의하는 훅으로, 데이터 fetch, 이벤트 구독, DOM 수정 같은 비동기·외부 의존 로직을 안전하게 처리할 수 있게 한다.

① 구조

```
useEffect(() => {  
  // 실행할 코드  
  return () => {  
    // 정리(clean-up) 코드  
  };  
}, [의존성]);
```

② 특징

- 렌더링 이후 동작

- 정리 함수로 메모리 누수 방지 가능
- 비동기 데이터 fetch에 최적

컴포넌트 로드 시 데이터 가져오기

```
useEffect(() => {
  fetch("https://jsonplaceholder.typicode.com/posts/1")
    .then(res => res.json())
    .then(data => setPost(data));
}, []);
```

(4) 의존성 배열 개념과 라이프사이클 비교

useEffect의 두 번째 인자인 의존성 배열을 통해 실행 시점을 제어할 수 있으며, 빈 배열은 마운트 시 1회만 실행, 값 지정은 특정 값 변경 시 실행, 생략은 모든 렌더링마다 실행되어 클래스 컴포넌트의 생명주기와 비교 가능하다.

- `[]` (빈 배열): 마운트 시 1회 실행 (componentDidMount)
- `[value]`: 특정 값 변경 시 실행 (componentDidUpdate)
- 배열 생략: 매 렌더링마다 실행
- return의 clean-up: componentWillUnmount

① 윈도우 크기 이벤트 등록/해제

```
useEffect(() => {
  const handleResize = () => console.log(window.innerWidth);
  window.addEventListener("resize", handleResize);

  return () => window.removeEventListener("resize", handleResize);
}, []);
```

(5) useState & useEffect 실습 애플리케이션

이번 예제는 **API 데이터 fetch + 카운터 상태 관리**를 동시에 다루어 `useState` 와 `useEffect` 의 활용을 종합적으로 실습합니다.

① 디렉터리 구조

```
ex02-usestate-useeffect/  
├── public/  
│   └── index.html  
├── src/  
│   ├── App.js  
│   ├── Counter.js  
│   ├── PostViewer.js  
│   ├── index.js  
│   └── App.css  
└── package.json
```

② 실행 명령어

```
# 프로젝트 생성  
npx create-react-app ex02-usestate-useeffect  
cd ex02-usestate-useeffect  
  
# 필요 라이브러리 설치 (없으면 기본 CRA로 충분)  
npm install  
  
# 개발 서버 실행  
npm start
```

③ `src/index.js`

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";  
import "./App.css";
```

```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

④ src/App.js

```
import React from "react";
import Counter from "./Counter";
import PostViewer from "./PostViewer";

function App() {
  return (
    <div className="app-container">
      <h1>Ex02: useState & useEffect Demo</h1>
      <Counter />
      <hr />
      <PostViewer />
    </div>
  );
}

export default App;
```

⑤ src/Counter.js

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div className="counter">
      <h2>Counter Example</h2>
      <p>현재 값: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1 증가</button>
    </div>
  );
}
```

```

    <button onClick={() => setCount(count - 1)}>-1 감소</button>
    <button onClick={() => setCount(0)}>초기화</button>
  </div>
);
}

export default Counter;

```

⑥ src/PostViewer.js

```

import React, { useState, useEffect } from "react";

function PostViewer() {
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts/1")
      .then((res) => res.json())
      .then((data) => {
        setPost(data);
        setLoading(false);
      });
  }, []);

  return (
    <div className="post-viewer">
      <h2>API Fetch Example</h2>
      {loading ? <p>로딩 중...</p> : (
        <div>
          <h3>{post.title}</h3>
          <p>{post.body}</p>
        </div>
      )}
    </div>
  );
}

```

```
export default PostViewer;
```

⑦ src/App.css

```
.app-container {  
  font-family: Arial, sans-serif;  
  padding: 20px;  
}  
  
.counter, .post-viewer {  
  margin: 20px 0;  
}  
  
button {  
  margin-right: 10px;  
  padding: 5px 10px;  
}
```

- `useState` : 카운터 증가/감소/초기화
- `useEffect` : API 데이터 fetch 후 화면 반영
- `의존성 배열 []` : 마운트 시 1회만 실행

4.3 Context API를 통한 상태관리

Context API는 전역 상태 공유를 위한 내장 기능으로, props drilling 문제를 해결하고 `createContext`와 `useContext`로 손쉽게 컨텍스트를 정의·사용하며, `Provider`와 `Consumer`를 통해 상태를 계층적으로 전달하고, 실제 실습에서는 인증 상태나 테마와 같은 공통 데이터를 전역 관리 구조로 구현할 수 있다.

(1) 전역 상태의 필요성과 구조

규모가 커진 애플리케이션에서 다수의 컴포넌트가 동일한 데이터를 필요로 할 때, 전역 상태를 두어 데이터의 일관성을 유지하고 중첩된 props 전달의 비효율을 줄이는 구조가 필요하다.

- `props drilling` : 데이터가 꼭 필요 없는 중간 컴포넌트까지 전달됨
- 전역 상태 관리: Context API, Redux, Recoil 등의 방법 존재
- Context API는 React 내장 기능으로 추가 라이브러리 없이 가능

(2) createContext, useContext 및 구조

`createContext`로 컨텍스트 객체를 생성하고, `useContext` 혹은 사용하여 원하는 컴포넌트에서 해당 컨텍스트 값을 직접 참조함으로써 간단하고 선언적인 전역 상태 사용 구조를 구현한다.

```
const ThemeContext = createContext("light");
```

- `createContext` : 컨텍스트 객체 생성, 기본값 지정 가능
- `useContext` : 함수형 컴포넌트 내에서 Context 값 접근

```
const theme = useContext(ThemeContext);
```

(3) Provider, Consumer 구조 실습

`Context.Provider`를 사용하여 전역 상태와 값을 하위 컴포넌트에 공급하고, `Consumer` 또는 `useContext`로 이를 구독하는 구조를 실습함으로써 상태 전달의 원리를 체득할 수 있다.

- `Context.Provider` : value 속성으로 전달
- 모든 하위 컴포넌트에서 접근 가능
- `Consumer` 방식도 가능하지만, 최신 코드에서는 `useContext` 를 권장

(4) 계층형 데이터 전달 예제 구성

다중 Provider를 중첩 배치하여 사용자 인증, UI 테마, 언어 설정과 같은 서로 다른 전역 상태를 계층적으로 관리하고, 복잡한 데이터 흐름을 구조화하는 예제를 구성할 수 있다.

- 예: `AuthContext`, `ThemeContext`, `LanguageContext`
- Provider를 트리 구조로 배치
- 각 Consumer 컴포넌트는 필요한 데이터만 구독

(5) Context API 실습 애플리케이션

이번 예제에서는 **테마 전환 (light/dark)**과 **사용자 인증 상태**를 Context API로 관리합니다.

① 디렉터리 구조

```
ex03-context-api/  
├── public/  
│   └── index.html  
├── src/  
│   ├── contexts/  
│   │   ├── ThemeContext.js  
│   │   └── AuthContext.js  
│   ├── components/  
│   │   ├── Header.js  
│   │   ├── ThemeToggler.js  
│   │   └── UserProfile.js  
│   ├── App.js  
│   ├── index.js  
│   └── App.css  
└── package.json
```

② 실행 명령어

```
# 프로젝트 생성
npx create-react-app ex03-context-api
cd ex03-context-api

# CRA 기본 패키지로 충분
npm install

# 실행
npm start
```

③ `src/index.js`

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

④ `src/contexts/ThemeContext.js`

```
import { createContext, useState } from "react";

export const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
    setTheme((prev) => (prev === "light" ? "dark" : "light"));
  };

  return (
```



```

    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

```

⑤ `src/contexts/AuthContext.js`

```

import { createContext, useState } from "react";

export const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (name) => setUser({ name });
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

```

⑥ `src/components/Header.js`

```

import React, { useContext } from "react";
import { ThemeContext } from "../contexts/ThemeContext";
import { AuthContext } from "../contexts/AuthContext";

function Header() {
  const { theme } = useContext(ThemeContext);
  const { user } = useContext(AuthContext);

  return (

```

```

    <header className={`header ${theme}`}>
      <h1>Ex03: Context API Demo</h1>
      <p>{user ? `환영합니다, ${user.name}님!` : "로그인하지 않았습니다."}</p>
    </header>
  );
}

export default Header;

```

⑦ src/components/ThemeToggler.js

```

import React, { useContext } from "react";
import { ThemeContext } from "../contexts/ThemeContext";

function ThemeToggler() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <div>
      <p>현재 테마: {theme}</p>
      <button onClick={toggleTheme}>테마 전환</button>
    </div>
  );
}

export default ThemeToggler;

```

⑧ src/components/UserProfile.js

```

import React, { useContext } from "react";
import { AuthContext } from "../contexts/AuthContext";

function UserProfile() {
  const { user, login, logout } = useContext(AuthContext);

  return (

```

```

    <div>
      {user ? (
        <>
          <p>사용자: {user.name}</p>
          <button onClick={logout}>로그아웃</button>
        </>
      ) : (
        <><p>로그인 필요</p>
          <button onClick={() => login("홍길동")}>로그인</button>
        </>
      )}
    </div>
  );
}

export default UserProfile;

```

⑨ src/App.js

```

import React from "react";
import { ThemeProvider } from "../contexts/ThemeContext";
import { AuthProvider } from "../contexts/AuthContext";
import Header from "../components/Header";
import ThemeToggler from "../components/ThemeToggler";
import UserProfile from "../components/UserProfile";

function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <div className="app">
          <Header />
          <ThemeToggler />
          <UserProfile />
        </div>
      </AuthProvider>
    </ThemeProvider>
  );
}

```

```
);
}

export default App;
```

⑩ src/App.css

```
.app {
  font-family: Arial, sans-serif;
  padding: 20px;
}

.header {
  padding: 10px;
  margin-bottom: 20px;
}

.header.light {
  background-color: #f0f0f0;
  color: #333;
}

.header.dark {
  background-color: #333;
  color: #f0f0f0;
}

button {
  margin-top: 10px;
  padding: 5px 10px;
}
```

- `createContext`, `useContext` 활용
- `Provider` 로 전역 상태 공급
- 테마(light/dark) 전환
- 사용자 로그인/로그아웃 전역 관리

- `props drilling` 없이 필요한 컴포넌트에서 바로 사용